Machine Learning

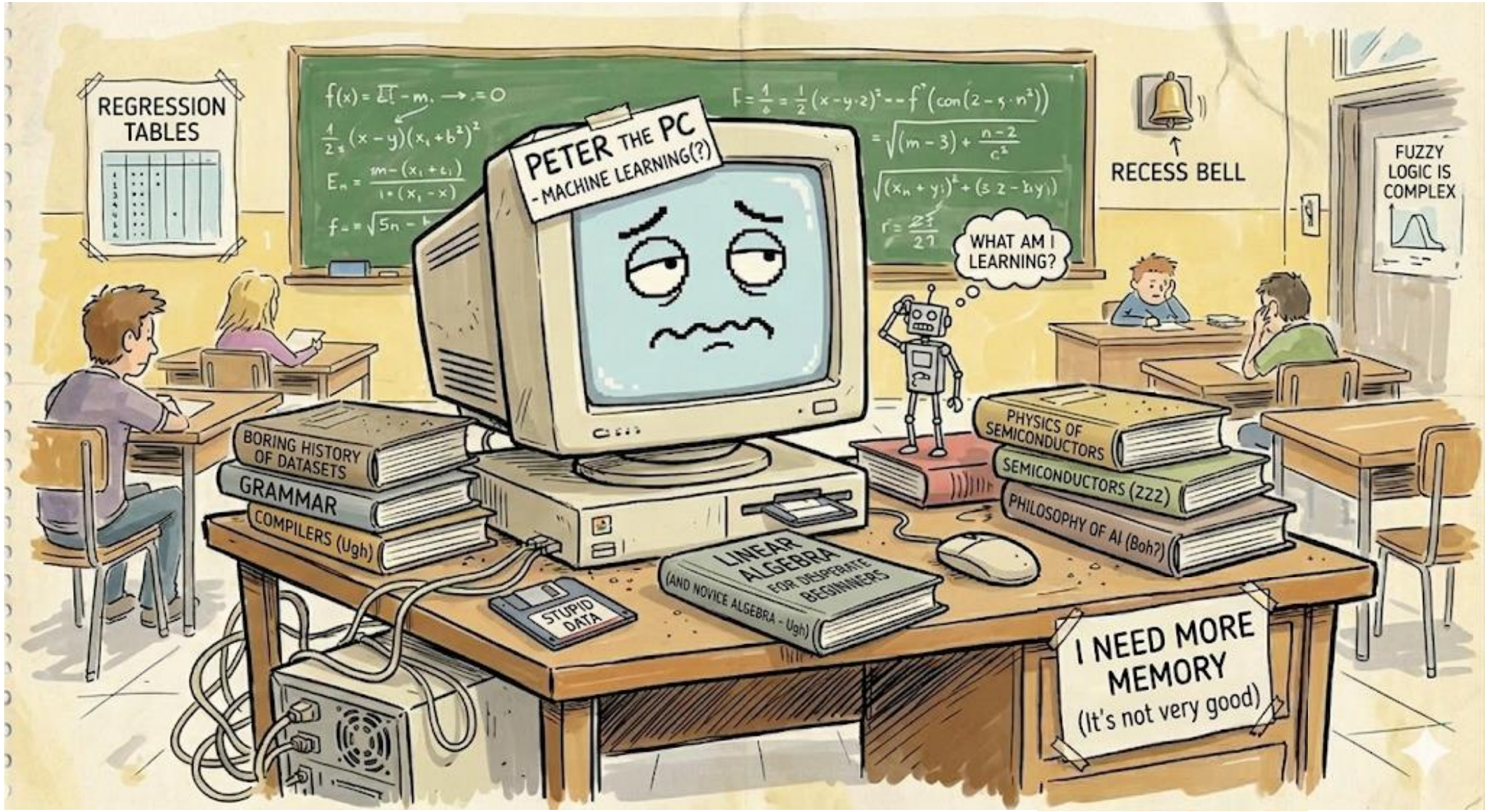
Irene Siragusa, PhD

CHILAB
Università degli Studi di Palermo

Outline

- Introduction
- Typology of learning tasks
- Data usage
- Performance, capacity and generalization
- Training techniques

Machine Learning?


REGRESSION TABLES
PETER THE PC
- MACHINE LEARNING(?)
RECESS BELL
FUZZY LOGIC IS COMPLEX
WHAT AM I LEARNING?
BORING HISTORY OF DATASETS
GRAMMAR
COMPILERS (Ugh)
PHYSICS OF SEMICONDUCTORS
SEMICONDUCTORS (ZZZ)
PHILOSOPHY OF AI (Boh?)
LINEAR ALGEBRA FOR DESPERATE BEGINNERS
(AND NOVICE ALGEBRA - Ugh)
STUPID DATA
I NEED MORE MEMORY
(It's not very good)

Introduction

- Machine Learning (ML) is essentially a form of applied statistics with increased emphasis on the use of computers to statistically estimate complicated functions and a decreased emphasis on proving confidence intervals around these functions.

- ML is the development of computational models for automatic learning over a data set.

Introduction

- ML algorithm is an algorithm that is able to learn from data.
- A computer program is said to learn from experience *E* with respect to someclass of tasks *T* and performance measure *P*, if its performance at tasks in *T*, as measured by *P*, improves with experience (*Mitchell, 1997)*
- Learning process implies an increasing in performance *P* over a target task *T* as the experience *E* increases.

Introduction

A learning algorithm or a model, must provide a statistical estimator of a complex function that connects data with the expected output

$$\boldsymbol{y} = f(\boldsymbol{x}, \boldsymbol{w}; \theta)$$

Introduction

A learning algorithm or a model, must provide a statistical estimator of a complex function that connects data with the expected output

Model: characterization of the family functions *f* and/or for the used algorithm

$$\boldsymbol{y} = f(\boldsymbol{x}, \boldsymbol{w}; \theta)$$

Introduction

A learning algorithm or a model, must provide a statistical estimator of a complex function that connects data with the expected output

Model: characterization of the family functions *f* and/or for the used algorithm

Model parameter: quantities involved in *f* function and should be learned from data

$$\boldsymbol{y} = f(\boldsymbol{x}, \boldsymbol{w}; \theta)$$

Introduction

A learning algorithm or a model, must provide a statistical estimator of a complex function that connects data with the expected output

Model: characterization of the family functions *f* and/or for the used algorithm

Model parameter: quantities involved in *f* function and should be learned from data

$$\boldsymbol{y} = f(\boldsymbol{x}, \boldsymbol{w}; \theta)$$

Hyperparameters: quantites that condition the evolution of the algorithm during the training process. They should be fixed during training process, and can be estimated leveraging data, but with different techniques as for the train of the model

Typology of learning tasks

- Clustering
  - The algorithm divides the data set into *k* partitions so that each partition (or cluster) contains data that is much more similar to each other than the data belonging to other partitions

$$f : \mathbb{R}^n \rightarrow \{1, \ldots, k\}$$

  - Partitions emerge from the internal structure of the data

Typology of learning tasks

- Classification
  - The algorithm estimates a function that makes each input correspond to one of *k* distinct classes

$$f : \mathbb{R}^n \rightarrow \{1, \ldots, k\}$$

  - The difference with clustering is that classes represent an exogenous process on the data, i.e. are not inferred, are known *a priori*
  - Classification function *f* may output a probability distribution over *k* classes

Typology of learning tasks

- Regression
  - Prediction of a numerical value from inputs
  - Similar to classification, but the function $f : \mathbb{R}^n \rightarrow \mathbb{R}$

Typology of learning tasks

- Transcription
  - Observation of poorly structured data and transcription in textual form
  - Optical Character Recognition (OCR): from images to text
  - Speech recognition: from sound to text
  - Can be considered as classification

Typology of learning tasks

- Machine translation
  - Converting one discrete sequence of symbols to another
  - Natural Language Processing (NLP) task

Typology of learning tasks

- Structured output tasks
  - Machine Learning tasks where output is a vector or another structure in which data are highly correlated
  - Generated output is made up of several interrelated values
  - NLP tasks such as parsing or QA
  - Segmentation and image captioning

classifications

Typology of learning tasks

- Anomaly detection
  - Identification of inputs such as to be exceptional or atypical with respect to a given *normal* behavior of the population samples
  - Can be considered as clustering and/or classification interpreted in dual form, i.e. looking for elements *not belonging* to the normal class
  - e.g. in credit card fraud detection, frauds are outliers from user's purchase habits and the corresponding probability distribution function

Typology of learning tasks

- Synthesis and sampling
  - Generating samples similar to input data
  - Multimedia applications
  - Automatic texture generation in video games
  - Speech synthesis as for vocal assistants

Typology of learning tasks

- Missing data inputation
  - Prediction of missing features from incomplete input
- Denoising
  - Prediction of $\mathbf{x} \in \mathbb{R}^n$, considering $\mathbf{x}'$ as corrupted version of $\mathbf{x}$
  - $p(\mathbf{x} \mid \mathbf{x}')$

Typology of learning tasks

- Estimating a probability density or mass function
  - This is the task of estimating the function

$$p_{\text{model}} : \mathbb{R}^n \rightarrow \mathbb{R}$$

which can be considered as a probability density function $p(\mathbf{x})$, or a mass function as for discrete variables

  - It is used to obtain an explicit estimate of $p(\mathbf{x})$ that is used for other learning tasks

Data usage

- Unsupervised learning
  - The objective is to learn a probability distribution function $p(\mathbf{x})$ which underlies the given data set
  - Generally speaking, the focus is towards learning data's structure
  - e.g. clustering, denoising

Data usage

- Supervised learning
  - Each sample **x** of the input data is associated with a textual or numeric label **y**
  - The algorithms learns how to predict **y** with **x** given, thus resulting in predicting $p(\mathbf{y} \mid \mathbf{x})$
  - e.g. classification, regression

Data usage

- Reinforcement learning
  - The data set is not fixed and the algorithm interacts with the environment, learning through a trial and error procedure
  - The algorithms (or better the agent) at time t is in state S_t and executes an action A_t obtaining a reward R_t
  - The agent evolves along a state-action-reward cycle

Data usage

- Reinforcement learning
  - Global reward G_t at time *t* is the sum of all the possible reward values linked to future behaviours, at least up to a certain time horizon of *T* instants of time in the future.

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \text{ where } \gamma \in [0, 1)$$

  - γ it is the discount factor that weighs less and less on actions carried out in the more distant future

Data usage

- Reinforcement learning – policy function usage
  - The agent uses a policy function π for choosing the action to be taken given the current state s: $a = \pi(s)$
  - Agent's objective is to maximize the reward values, as expected values from the policy functon:

$$v_\pi(s) = \mathbb{E}_\pi \left[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots \mid S_t = s \right]$$

  - The maximixation is directly towars π function, eventually defined in a probabilistic manner as $\pi(a \mid s) = P[A_t = a \mid S_t = s]$

Data usage

- Reinforcement learning – Q Learning
  - A ML model is trained to maximize a quality function $Q(s_t, a_t)$ obtained if at time t a given action a_t is done at the current state s_t:

$$Q(s_t, a_t) \leftarrow \underbrace{Q(s_t, a_t)}_{\text{old value}} + \underbrace{\alpha_t(s_t, a_t)}_{\text{learning rate}} \times \left[\overbrace{\underbrace{R_{t+1}}_{\text{reward}} + \underbrace{\gamma}_{\text{discount factor}} \underbrace{\max_{a_{t+1}} Q(s_{t+1}, a_{t+1})}_{\text{maximum future value}}}^{\text{learned value}} - \underbrace{Q(s_t, a_t)}_{\text{old value}} \right]$$

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t)(1 - \alpha_t(s_t, a_t)) + \alpha_t(s_t, a_t)[R_{t+1} + \gamma \max_{a_{t+1}} Q(s_{t+1}, a_{t+1})]$$

Performance, capacity and generalization

- The main objective of a learning algorithm is to generalize from the observed data, i.e. to *make correct predictions over unobserved data*.

- To reach the objective, each algorithm leverages on a suitable performance measure *P* that *minimizes the error in prediction or inference phase*.

Performance, capacity and generalization

- A quantitative measure of performance of a machine learning algorithm must be designed to evaluate its abilities.
- This performance measure P is specific to the task T being carried out by the system.
- Accuracy of the model: proportion of examples for which the model produces the correct output.
- Error rate, the proportion of examples for which the model produces an incorrect output.

Performance, capacity and generalization

- Each algorithm must minimize two errors measures:
  - Training error, the error measure over training data
  - Test error or generalization error, the error measure of testing data, mean to test the goodness of the performance and they *are absolutely distinct from those of training*

Performance, capacity and generalization

- The functional form *L* of those error measures, called loss function, depends on the specific algorithm used and on the conducted type of statistical estimate.

- 0-1 values may be used for error rate in discrete context.
- MSE can be used to establish the difference between real value and predicted one.
- The most common approach is to report the average log-probability the model assigns to some examples.

Performance, capacity and generalization

- The choice of performance measure is often difficult, since it has to corresponds well to the desired behavior of the system.
- It is difficult to decide what should be measured, and it depends on the application

Performance, capacity and generalization

- *Transcription task*, should we measure the accuracy of the system at transcribing entire sequences, or should we use a more fine-grained performance measure that gives partial credit for getting some elements of the sequence correct?

- *Regression task*, should we penalize the system more if it frequently makes medium-sized mistakes or if it rarely makes very large mistakes

Performance, capacity and generalization

- The ability to generalize is based on Statistical Learning Theory
  - There is a unique statistical process that describes the phenomenon under investigation and generates not only the data used for training, but also the data never seen
  - This process is described by a unique probability distribution p_{data}, not known a priori
  - An a priori assumption about the data is established, which is the foundation of any ML algorithm: the training and test data are independent and identically distributed (i.i.d.) from a given probability distribution.

Performance, capacity and generalization

- The i.i.d. hypothesis tells that, for any model, the expected value of the training error and the test error are equal
  - There is certainly a configuration of the parameters ***w*** for which these two errors are equal
  - In reality, because of the sampling of available data, this condition does not occur

Performance, capacity and generalization

- The i.i.d. hypothesis tells that, for any model, the expected value of the training error and the test error are equal
  - The algorithm tries to minimize the training error through training on the training set, which is a loss minimization procedure
  - The algorithm tries to keep minimal the gap with the generalization error

Performance, capacity and generalization

- Overfitting
  - Excess minimization of training error, resulting in poor generalization
  - High generalization error
- Underfitting
  - Low ability to minimize training error

Performance, capacity and generalization

- The capacity of a model is its ability to approximate the widest possible range of functional forms
- The range of functional forms directly obtainable from the model is referred to as hypothesis space
- The capacity of the model can be manageable through its hyperparameters

Performance, capacity and generalization

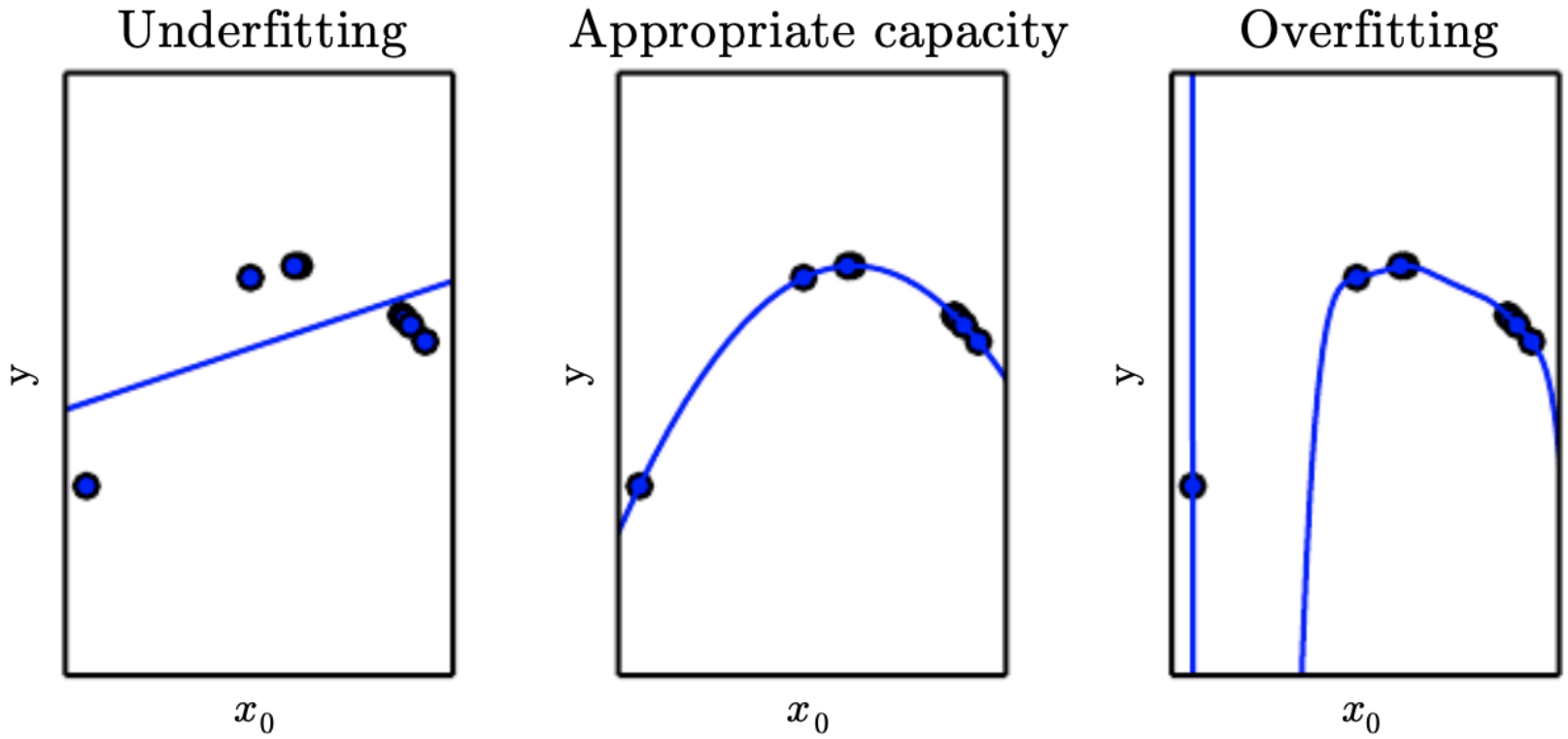
- e.g. regression problem

$$y = \boldsymbol{w} \cdot \boldsymbol{x} + b,$$ linear regression

$$y = \sum_{i=1}^{n} \boldsymbol{w}_i \cdot \boldsymbol{x}^i + b,$$ polynomial regression

  - Hypothesis space → polynomial functions
  - Pameters → {b, $\boldsymbol{w}_i$ i = 1, …, n}
  - Hyperpameters → n

Performance, capacity and generalization



$$\hat{y} = b + wx \qquad \hat{y} = b + w_1 x + w_2 x^2 \qquad \hat{y} = b + \sum_{i=1}^{9} w_i x^i.$$

Performance, capacity and generalization

- Machine learning algorithms will generally perform best when their capacity is appropriate for the true complexity of the task they need to perform and the amount of training data they are provided with.
- Models with insufficient capacity are unable to solve complex tasks.
- Models with high capacity can solve complex tasks, but when their capacity is higher than needed to solve the present task, they may overfit.

Performance, capacity and generalization

- Capacity is not determined only by the choice of model
  - Representational capacity: the model specifies which family of functions the learning algorithm can choose from when varying the parameters to reach a training objective.

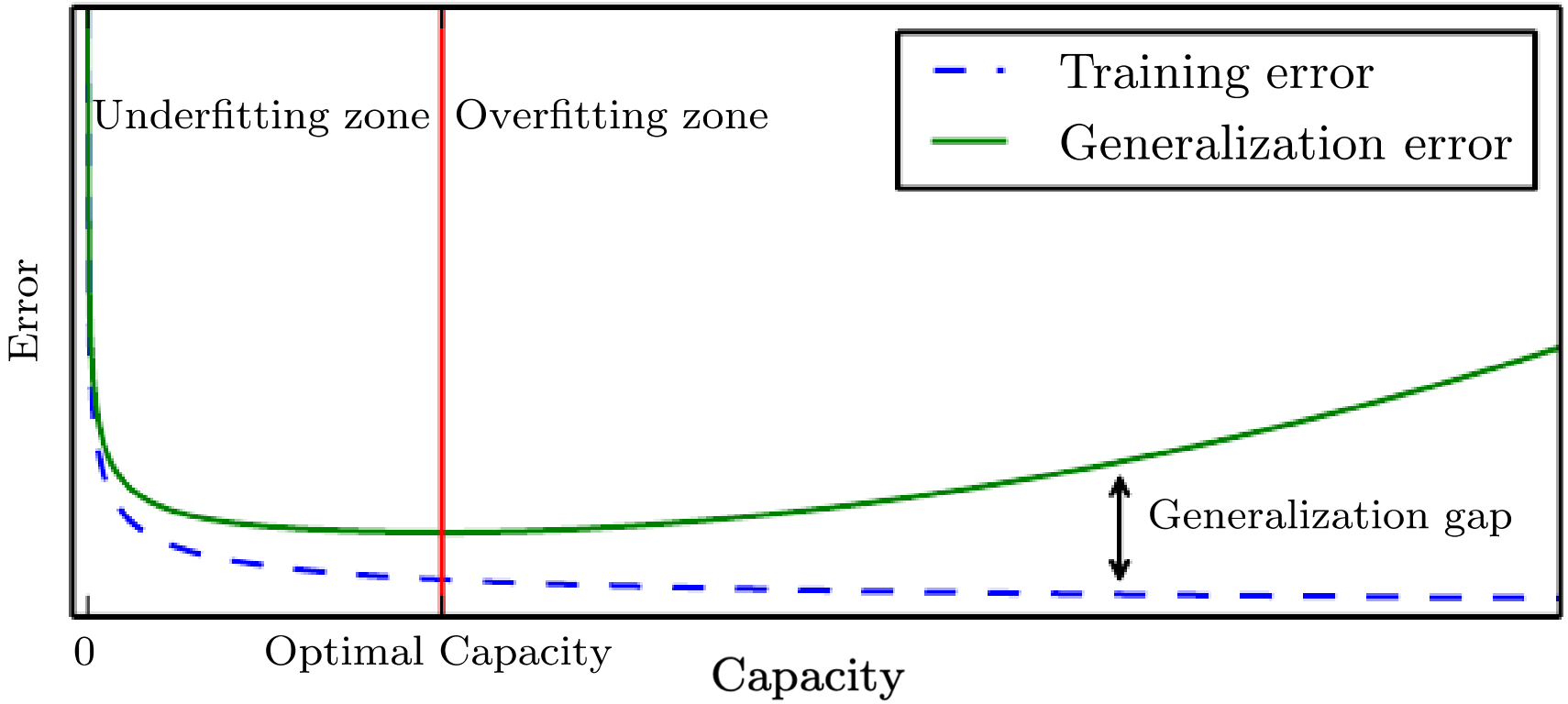
Performance, capacity and generalization

- Finding the best function within this family is a difficult optimization problem
  - The learning algorithm does not actually find the best function, but merely one that significantly reduces the training error.
  - Additional limitations, such as the imperfection of the optimization algorithm, mean that the learning algorithm's effective capacity may be less than the representational capacity of the model family.

Performance, capacity and generalization

- Occam's razor
  - Principle that states that among competing hypotheses that explain known observations equally well, we should choice the "simplest" one.
- This idea is behind the statistical learning theory:
  - Quantifying the capacity of the model enables statistical learning theory to make quantitative predictions.
  - The discrepancy between training error and generalization error is bounded from above by a quantity that grows as the model capacity grows but shrinks as the number of training examples increases.

Performance, capacity and generalization



*Training error decreases until it asymptotes to the minimum possible error value as model capacity increases**

**assuming the error measure has a minimum value*

Performance, capacity and generalization

- Parametric models learn a function described by a parameter vector whose size is finite and fixed before any data is observed.

- Nonparametric models have no such limitation, sometimes they are just theoretical abstractions that cannot be implemented in practice.
  - such as an algorithm that searches over all possible probability distributions

Performance, capacity and generalization

- It is possible to design practical nonparametric models by making their complexity a function of the training set size or create a nonparametric learning algorithm by wrapping a parametric learning algorithm inside another algorithm that increases the number of parameters as needed.

Performance, capacity and generalization

- The ideal model is an oracle that simply knows the true probability distribution that generates the data.
  - Errors will occur due to some noise in the distribution.
  - Error incurred by an oracle making predictions from the true distribution is called the Bayes error.

Performance, capacity and generalization

- The training error and generalization of a learning algorithm also depends on the size from the training set

- In principle, an *oracle* that knows exactly p_{data} commits a constant prediction error known as Bayes error

- The oracle does not know *exactly* the data, but its probability distribution

Performance, capacity and generalization

- For nonparametric models, more data yield better generalization until the best possible error is achieved.
- Any fixed parametric model with less than optimal capacity will asymptote to an error value that exceeds the Bayes error.
- It is possible for the model to have optimal capacity and yet still have a large gap between training and generalization errors. This gap may be solved by gathering more training examples.

Performance, capacity and generalization

- Learning theory claims that a machine learning algorithm can generalize well from a finite training set of examples.
  - Inductive reasoning or inferring general rules from a limited set of examples, is not logically valid.
  - To logically infer a rule describing every member of a set, one must have information about every member of that set.

- Machine learning avoids (partially) this problem:
  - It offers only probabilistic rules, rather than the entirely certain rules used in purely logical reasoning.
  - Its objective is to find rules that are probably correct about most members of the set they concern.

Performance, capacity and generalization

- No free lunch theorem
  - The performance of any learning algorithm, averaged over any probability distribution that generates the data, *is the same*
  - There are no algorithms in principle better than others
  - In fact, the design of a model based on particular assumptions made about p_{data}, i.e., the careful choice of the hypothesis space, makes certain algorithms better than others on that particular form of data generation process

Performance, capacity and generalization

- No free lunch theorem
  - If we make assumptions about the kinds of probability distributions we encounter in real-world applications, then we can design learning algorithms that perform well on these distributions.
  - The goal of machine learning research *is not* to seek a universal learning algorithm or the absolute best learning algorithm.
  - Instead, our goal is to understand what kinds of distributions are relevant to the *real world* that an AI agent experiences, and what kinds of machine learning algorithms perform well on data drawn from the kinds of data-generating distributions we care about.

ML algorithms must be designed to perform well on a specific task

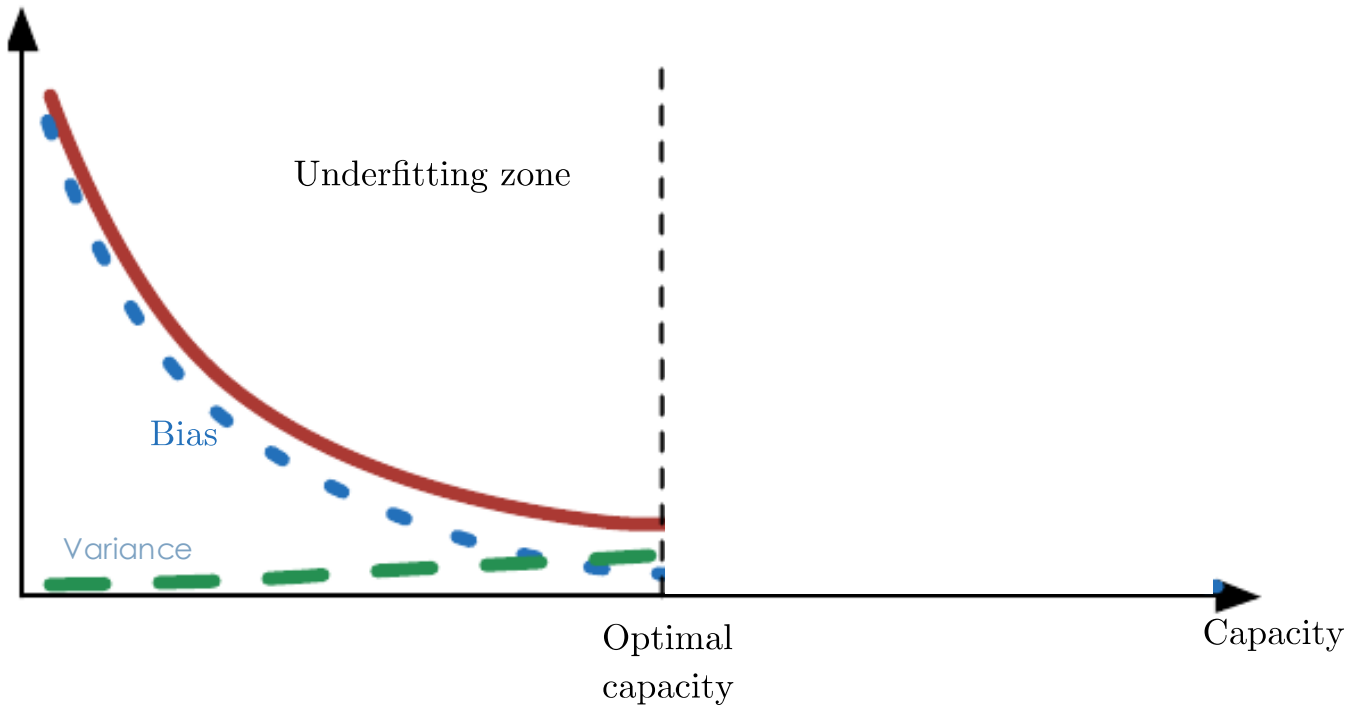
Performance, capacity and generalization

Regression

- The regression algorithm learns by minimizing the MSE_{train} computed on the training set, keeping the gap from the generalization error MSE_{test} minimal
- This corresponds to an MLE estimate of the optimal parameters of $p_{\text{model}}(y \mid \mathbf{x}_{\sim \text{pdata}})$ assumed Gaussian
- We analyze the goodness of the estimate in terms of the capacity

Performance, capacity and generalization

$$\mathrm{MSE} = \mathbb{E}\left[(\hat{\theta}_m - \theta)^2\right] = \mathrm{Bias}(\hat{\theta}_m)^2 + \mathrm{Var}(\hat{\theta}_m)$$

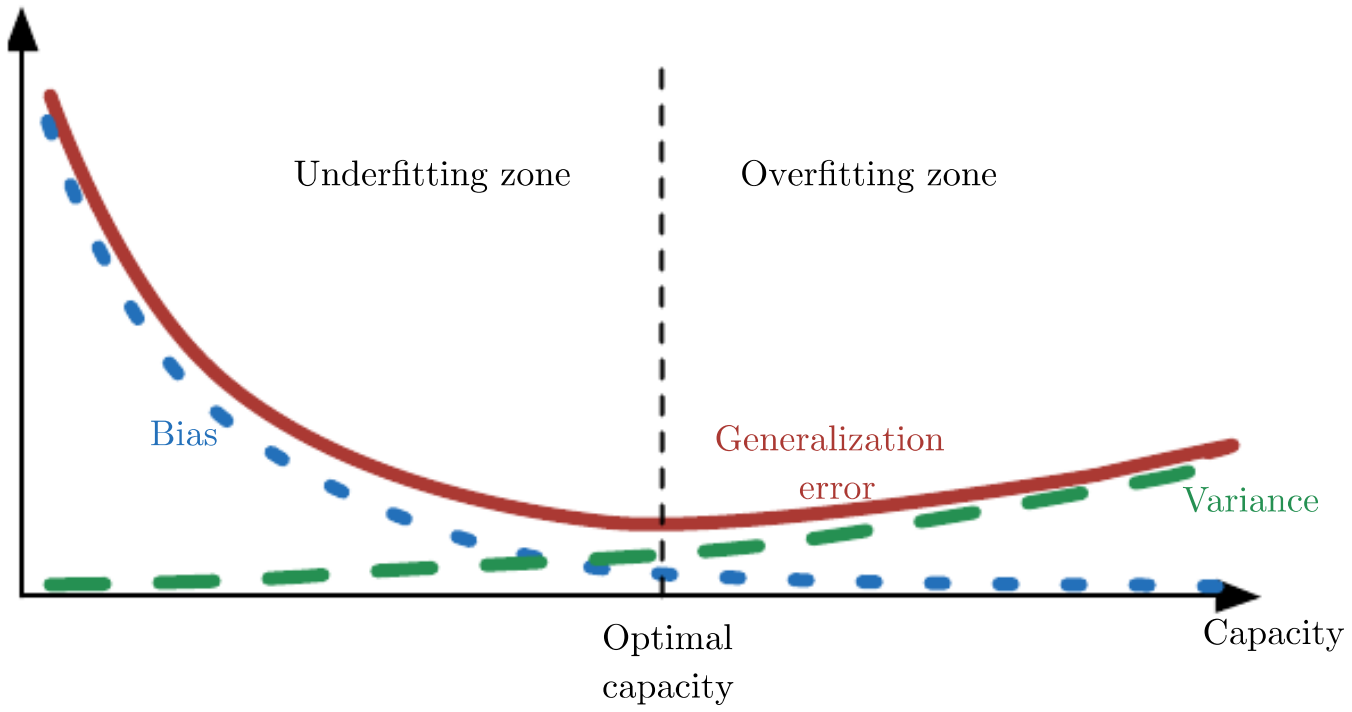


Underfitting

- training set is better learned as the capacity increaes
- Bias (expected values over training set) decreases
- Variance (due to test set) does not increases a lot

Performance, capacity and generalization

$$\mathrm{MSE} = \mathbb{E}\left[(\hat{\theta}_m - \theta)^2\right] = \mathrm{Bias}(\hat{\theta}_m)^2 + \mathrm{Var}(\hat{\theta}_m)$$

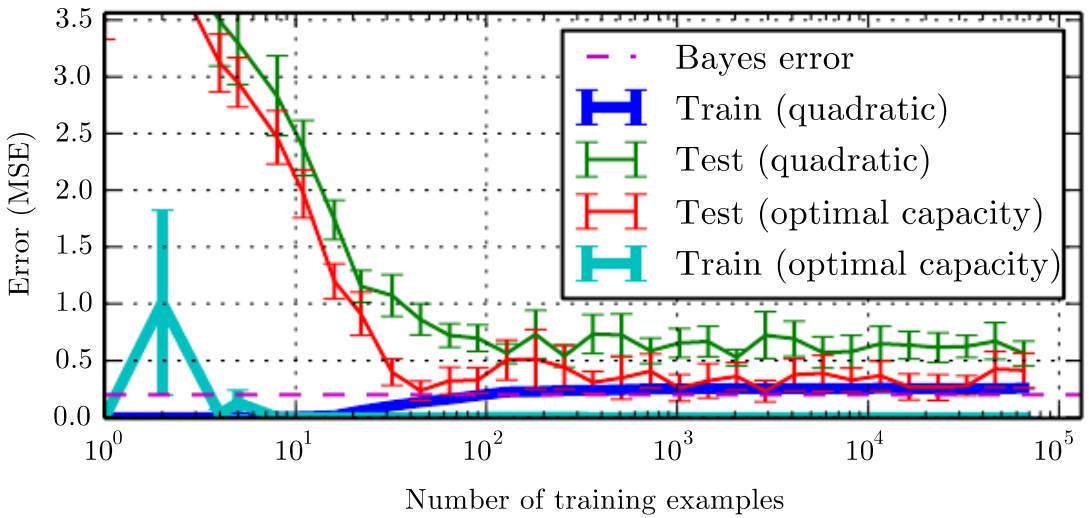


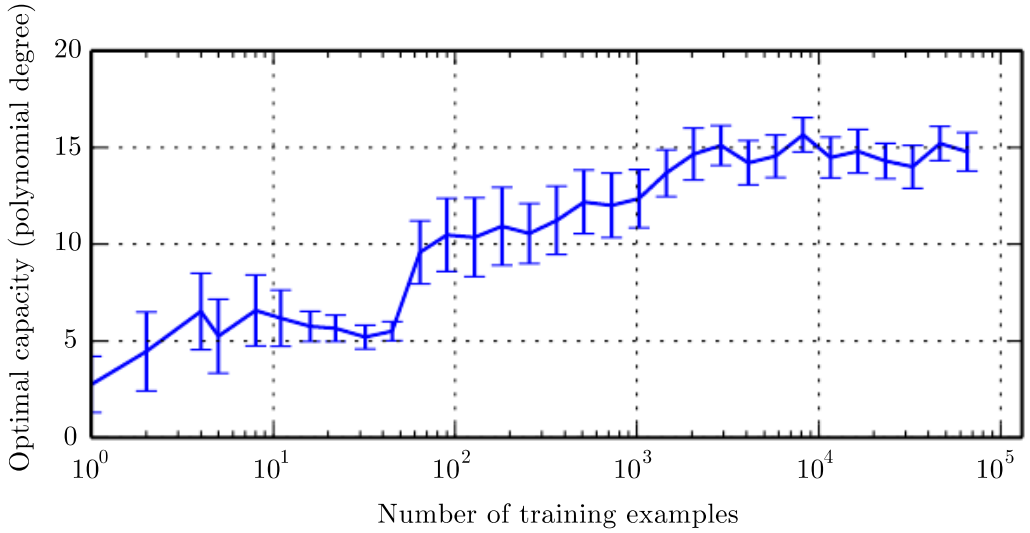
Overfitting

- training set is better learned as the capacity increaes
- Bias over training set becomes minimal
- Variance (due to test set) becomes very high

Performance, capacity and generalization

- As the training data increases, generalization error asymptotically tends to a value that is greater (at low model capacity) or equal to the Bayes error





Synthetic data set generated by adding noise to points obtained from a polynomial of degree 5

Training techniques

- Learning algorithms use several techniques that modify standard training procedures in order to increase the *effective capability* of the chosen model

- These techniques directly impact the choice of hyperparameters on which the (theoretical) representation capability of the model depends

  - Regolarization
  - Validation set
  - Cross-validation

Training techniques

- Hyperparameters
  - Parametric settings that can be used to control the algorithm's behavior.
  - Their values are not adapted by the learning algorithm itselt, *but* we can design a nested learning procedure in which one learning algorithm learns the best hyperparameters for another learning algorithm).
  - Capacity, epochs, learning strategy, architectural choices, …

Training techniques

- Regolarization
  - Any modification of the learning algorithm aimed at explicitly reducing the generalization error, but not the training error
  - The regularization term allows us to choose some functional forms over others in our hypothesis space
  - As in the no free lunch theorem, there is also no best form of regularization.
  - The *best* a form of regularization is the one well suited to the particular task intended to solve.

Training techniques

- Regolarization
  - weight decay regularization for regression, express its preference towards lower $\boldsymbol{w}$ values

λ is another hyperparameter

$$\mathcal{L}(\boldsymbol{w}) = \mathrm{MSE}_{\mathrm{train}} + \lambda \boldsymbol{w}^\top \boldsymbol{w}$$

  - A typical effect of regularization is the smoothing of the loss function whereby, analytically, $\boldsymbol{\mathcal{L}}$ converges to a global minimum other than the pure MSE

Training techniques

- Validation set
  - The validation set is used to overcome the problem of *learning hyperparameters*
  - The validation set is a portion of the training set that is expunged from the training set before the actual training begins
    - 80% of the training data for training and 20% for validation

Training techniques

- Validation set
  - The validation set is used to check the trend of generalization error by modifying the hyperparameters to minimize it.
  - The error committed on the validation set during training turns out to be a more correct estimate of the generalization error than the training error.

Training techniques

- Validation set
  - The validation set error will underestimate the generalization error, though typically by a smaller amount than the training error does.
  - After all hyperparameter optimization is complete, the generalization error may be estimated using the test set.

Training set → learning parameters

Validation set → learning hyperparameters

Training techniques

- Validation set
  - Training procedure outputs the best model according to the validation set
  - *A*: algorithm
  - $\mathbf{x}^{(\text{train})}$: training data
  - *split_perc*: split percentage
  - *H*: hyperparameters search space

Algorithm 1 Full training procedure

Input: $A, \boldsymbol{x}^{(train)}, split_perc, H$

$\boldsymbol{w} \leftarrow$ init_weights() {initialize the parameters as either zero or random values}

$L \leftarrow \infty,\ L^{val} \leftarrow \infty$ {initial values of the loss both for training and validation}

$(train_set, val_set) \leftarrow \text{split}(\boldsymbol{x}^{(train)}, split_perc)$ {split the training data}

for all $\theta \in H$ **do**

$(L, \boldsymbol{w}) \leftarrow A(train_set, \boldsymbol{w}, \theta)$ {training provides loss and weights}

if $L(val_set, \boldsymbol{w}, \theta) < L^{val}$ **then**

{save the state of the model related to the best validation error}

$L^{val} \leftarrow L(val_set, \boldsymbol{w}, \theta)$

$\theta^* \leftarrow \theta$

$\boldsymbol{w}^* \leftarrow \boldsymbol{w}$

$L^* \leftarrow L$

end if

end for

Output: $L^*, \boldsymbol{w}^*, \theta^*$ {output the best model as regards validation}

Training techniques

- Cross-validation
  - When there is a small number of samples, k-fold cross validation technique is adopted
  - The data set is divided into *k* partitions that do not overlap
  - A training strategy is adopted for each partition, whereby the partition acts as a test set and the rest as a training set

Training techniques

- Cross-validation
  - Within each training run, a validation set can be identified to train the model against its hyperparameters
  - The error committed is calculated as the *average value over all partitions* and reported as the model's performance over the entire data set
  - It is also possible to include cross-validation in the process of finding the best model hyperparameters

Training techniques

Algorithm 5.1 The k-fold cross-validation algorithm.

Define `KFoldXV`$(\mathbb{D}, A, L, k)$:
Require: $\mathbb{D}$, the given dataset, with elements $\boldsymbol{z}^{(i)}$
Require: A, the learning algorithm, seen as a function that takes a dataset as input and outputs a learned function
Require: L, the loss function, seen as a function from a learned function f and an example $\boldsymbol{z}^{(i)} \in \mathbb{D}$ to a scalar $\in \mathbb{R}$
Require: k, the number of folds
  Split $\mathbb{D}$ into k mutually exclusive subsets $\mathbb{D}_i$, whose union is $\mathbb{D}$
  for i from 1 to k **do**
    $f_i = A(\mathbb{D} \backslash \mathbb{D}_i)$
    for $\boldsymbol{z}^{(j)}$ in $\mathbb{D}_i$ **do**
      $e_j = L(f_i, \boldsymbol{z}^{(j)})$
    end for
  end for
  Return $\boldsymbol{e}$